LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack dan Queue

Oleh:

Siti Nafisah NIM. 2510817120009

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Siti Nafisah
NIM : 2510817120009

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL 1: Implementasi Stack 1
A. Source Code 1
B. Pembahasan 2
SOAL 2: Si Palui dan Mesin Hitung Aneh 5
A. Source Code 6
B. Output Program 8
C. Pembahasan 8
SOAL 3: Implementasi Queue 11
A. Source Code 11
B. Pembahasan 13
SOAL 4: Si Palui dan Deretan Angka 15
A. Source Code 16
B. Output Program 18
C. Pembahasan 18
TAUTAN GIT 20

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 2 8
Gambar 2. Screenshot Hasil Jawaban Soal 4 18

DAFTAR TABEL

Tabel 1 Source Code Stack.cpp 1
Tabel 2 Source Code Prob1.cpp 6
Tabel 3 Source Code Queue.cpp 11
Tabel 4 Source Code Prob2.cpp 16

SOAL 1: Implementasi Stack

Pada file **`stack.h`** yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file **`stack.cpp`**. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan **`throw`**.

Jelaskan alur algoritma dari setiap fungsi yang diim plementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1 Source Code Stack.cpp

```
#include "stack.h"

void init(Stack* s) {
    s->top = s->data;
    s->top--;
}

bool isEmpty(const Stack* s) {
    return s->top < s->data;
}

bool isFull(const Stack* s) {
    return s->top == s->data + MAX - 1;
}

void push(Stack* s, int value) {
    if (isFull(s)) {
```

```
        throw "stack penuh";
    }
    s->top++;
    *(s->top) = value;
}

void pop(Stack* s) {
    if (isEmpty(s)) {
        throw "stack kosong";
    }
    s->top--;
}

int peek(const Stack* s) {
    if (isEmpty(s)) {
        throw "stack kosong";
    }
    return *(s->top);
}
```

B. Pembahasan

Fungsi `init` digunakan untuk menginisialisasi stack agar dalam kondisi kosong. Caranya adalah dengan mengatur pointer `top` ke awal array (`data`), kemudian dikurangi satu posisi sehingga berada sebelum elemen pertama. Dengan kondisi ini, `top` belum menunjuk ke elemen mana pun sehingga stack dianggap kosong. Saat `push` pertama dilakukan, `top` akan naik ke posisi awal array. Hal ini menunjukkan bahwa kondisi awal stack selalu konsisten, yaitu kosong sebelum digunakan. Fungsi ini tidak memiliki kondisi tidak valid. Kompleksitas waktu O(1) karena hanya melakukan satu operasi penugasan, dan kompleksitas ruang O(1).

Fungsi `isEmpty` digunakan untuk mengecek apakah stack dalam keadaan kosong. Pengecekan dilakukan dengan membandingkan apakah `top` berada sebelum array `(top < data)`. Jika kondisi terpenuhi, berarti belum ada elemen yang masuk ke dalam stack. Hal ini sesuai karena sejak awal `top` diinisialisasi ke `data - 1`, dan hanya akan berubah jika ada operasi `push`. Fungsi ini tidak memiliki kondisi tidak valid. Kompleksitas waktu O(1) karena hanya melakukan satu perbandingan, dan kompleksitas ruang O(1).

Fungsi `isFull` digunakan untuk mengecek apakah stack sudah penuh. Hal ini dilakukan dengan memeriksa apakah `top` sudah berada di posisi terakhir array, yaitu `data + MAX - 1`. Jika sudah mencapai posisi tersebut, maka tidak ada lagi ruang untuk menambahkan elemen baru. Hal ini benar karena batas maksimum array sudah ditentukan, sehingga posisi tersebut merupakan batas akhir stack. Fungsi ini tidak memiliki kondisi tidak valid. Kompleksitas waktu O(1) dan ruang O(1).

Fungsi `push` digunakan untuk menambahkan elemen ke dalam stack. Langkah pertama adalah mengecek apakah stack penuh menggunakan `isFull`. Jika penuh, maka kondisi ini tidak valid dan program akan menghasilkan error (`throw`). Jika tidak penuh, maka pointer `top` dinaikkan satu posisi (`top++`) untuk menunjuk ke tempat kosong berikutnya, lalu nilai disimpan pada posisi tersebut (`*(s->top) = value`). Cara ini memastikan bahwa elemen baru selalu berada di atas elemen sebelumnya. Dengan demikian, urutan LIFO tetap terjaga, yaitu elemen terakhir yang masuk akan menjadi elemen pertama yang keluar. Kompleksitas waktu O(1) karena hanya terdiri dari beberapa operasi sederhana, dan kompleksitas ruang O(1).

Fungsi `pop` digunakan untuk menghapus elemen paling atas dari stack. Sebelum itu, dilakukan pengecekan apakah stack kosong menggunakan `isEmpty`. Jika kosong, maka kondisi tersebut tidak valid dan akan menghasilkan error (`throw`). Jika tidak kosong, maka pointer `top` diturunkan satu posisi (`top--`). Data sebenarnya tidak dihapus dari memori, tetapi dianggap keluar karena sudah tidak ditunjuk oleh `top`. Cara ini sesuai dengan prinsip LIFO, di mana elemen terakhir yang masuk adalah yang pertama keluar. Pengecekan kondisi kosong juga penting untuk mencegah terjadinya

pengambilan data dari stack yang tidak memiliki elemen. Kompleksitas waktu O(1) dan kompleksitas ruang O(1).

Fungsi `peek` digunakan untuk melihat nilai elemen paling atas tanpa menghapusnya. Sebelum mengambil nilai, dilakukan pengecekan apakah stack kosong. Jika kosong, maka kondisi tersebut tidak valid dan akan menghasilkan error. Jika tidak kosong, nilai yang ditunjuk oleh `top` dikembalikan. Hal ini benar karena `top` selalu menunjuk ke elemen terakhir yang dimasukkan ke dalam stack. Fungsi ini tidak mengubah isi stack sehingga hanya bersifat membaca data saja. Kompleksitas waktu O(1) dan kompleksitas ruang O(1).

SOAL 2: Si Palui dan Mesin Hitung Aneh

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi. Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- A_i adalah nilai bilangan pada input

- Simbol yang valid hanya:
  - bilangan bulat
  - operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

Tabel 2 Source Code Prob1.cpp

```
#include <iostream>
#include <string>
#include <cctype>
#include "stack.h"
using namespace std;

int main() {
```

```
    int n;
    cin >> n;

    Stack s;
    init(&s);

    for (int i = 0; i < n; i++) {
        string x;
        cin >> x;

       if (isdigit(x[0]) || (x[0] == '-' && x.length() > 1)) {
            push(&s, stoi(x));
        }
        else {
            int b = peek(&s);
            pop(&s);
            int a = peek(&s);
            pop(&s);

            int hasil = 0;
            if (x == "+") hasil = a + b;
            else if (x == "-") hasil = a - b;
            else if (x == "*") hasil = a * b;
            else if (x == "/") hasil = a / b;

            push(&s, hasil);
        }
    }

```

```
    cout << peek(&s) << endl;
    return 0;
}
```

B. Output Program

```
D:\Algoritma & Struktur Data\Modul 2>g++ prob1.cpp stack.cpp -o prob1

D:\Algoritma & Struktur Data\Modul 2>prob1
3
2 3 +
5

D:\Algoritma & Struktur Data\Modul 2>prob1
9
5 1 2 + 4 * + 3 -
14

D:\Algoritma & Struktur Data\Modul 2>prob1
5
10 2 / 3 *
15
```

Gambar 1. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Program ini digunakan untuk menghitung hasil yang terdiri dari angka dan operator dengan memanfaatkan struktur data stack. Di awal program terdapat `int n;` yang digunakan untuk mendeklarasikan variabel jumlah data. Kemudian pada baris berikutnya `cin >> n;` berfungsi untuk membaca banyaknya input yang akan diproses. Nilai `n` ini menentukan berapa kali perulangan akan dilakukan dalam program.

Selanjutnya pada bagian `Stack s;` dibuat sebuah variabel stack, dan pada baris `init(&s);` dilakukan inisialisasi agar stack berada dalam kondisi kosong. Hal ini penting supaya stack siap digunakan sebelum proses perhitungan dimulai.

Pada bagian perulangan `for (int i = 0; i < n; i++)`, program akan membaca input sebanyak `n` kali. Di dalam perulangan tersebut terdapat `string x;` dan `cin >> x;` yang digunakan untuk menyimpan input dalam bentuk string. Hal

ini bertujuan agar program bisa membedakan apakah input tersebut berupa angka atau operator.

Pada bagian `kondisi if (isdigit(x[0]) || (x[0] == '-' && x.length() > 1))`, program mengecek apakah input merupakan angka. Fungsi `isdigit` digunakan untuk mengecek apakah karakter pertama merupakan digit, sehingga diperlukan library `<cctype>` agar fungsi tersebut dapat digunakan. Jika kondisi ini terpenuhi, maka pada baris `push(&s, stoi(x));` nilai string akan dikonversi menjadi integer menggunakan `stoi`, lalu dimasukkan ke dalam stack menggunakan `push`.

Namun jika input bukan angka (berarti operator), maka pada bagian `else` program akan mengambil dua elemen teratas dari stack. Pada baris `int b = peek(&s); pop(&s);` nilai paling atas diambil dan disimpan ke variabel `b`, kemudian dihapus dari stack. Selanjutnya pada baris `int a = peek(&s); pop(&s);` diambil lagi elemen berikutnya dan disimpan ke variabel `a`. Urutan ini penting karena akan mempengaruhi hasil operasi.

Kemudian disiapkan variabel `hasil` untuk menyimpan hasil perhitungan. Setelah itu, pada bagian kondisi seperti `if (x == "+")`, `else if (x == "-")`, dan seterusnya, program akan melakukan operasi sesuai dengan operator yang dibaca. Hasil dari operasi tersebut disimpan ke dalam variabel `hasil`.

Pada baris `push(&s, hasil);` hasil perhitungan dimasukkan kembali ke dalam stack. Proses ini akan terus berulang sampai semua input selesai diproses.

Pada bagian akhir program, terdapat `cout << peek(&s)<< endl;` yang digunakan untuk menampilkan hasil akhir. Nilai yang diambil adalah elemen paling atas dari stack, karena setelah semua proses selesai, hasil akhir akan berada di posisi tersebut.

Sebagai contoh, jika pada input dimasukkan `n = 3` dengan data `2 3 +`, maka alurnya dimulai dengan memasukkan angka 2 dan 3 ke dalam stack. Ketika operator + dibaca, kedua nilai tersebut diambil dan dijumlahkan sehingga menghasilkan 5, lalu

dimasukkan kembali ke dalam stack. Pada akhir proses, nilai yang tersisa di dalam stack adalah 5, sehingga itulah hasil yang ditampilkan.

Secara keseluruhan, program ini bekerja dengan memanfaatkan stack untuk menyimpan sementara angka dan hasil operasi. Metode ini memudahkan proses perhitungan tanpa perlu memperhatikan prioritas operator secara manual. Kompleksitas waktu dari program ini adalah O(n) karena setiap input hanya diproses satu kali, sedangkan kompleksitas ruangnya adalah O(n) karena dalam kondisi terburuk semua elemen dapat disimpan di dalam stack.

SOAL 3: Implementasi Queue

Pada file **`queue.h`** yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file **`queue.cpp`**. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan **throw**.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 3 Source Code Queue.cpp

```
#include "queue.h"

void init(Queue* q) {
    q->front = q->data;
    q->rear = q->data;
}

bool isEmpty(const Queue* q) {
    return q->front == q->rear;
}

bool isFull(const Queue* q) {
    return (q->rear - q->data) == MAX;
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) {
```

```
        throw "queue penuh";
    }
    *(q->rear) = value;
    q->rear++;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) {
        throw "queue kosong";
    }

    for (int* p = q->front; p < q->rear - 1; ++p) {
        *p = *(p + 1);
    }

    q->rear--;
}

int front(const Queue* q) {
    if (isEmpty(q)) {
        throw "queue kosong";
    }
    return *(q->front);
}

int back(const Queue* q) {
    if (isEmpty(q)) {
        throw "queue kosong";
    }
    return *(q->rear - 1);
```

```
}
```

B. Pembahasan

Fungsi `init` digunakan untuk menginisialisasi queue agar dalam kondisi kosong. Pada bagian ini, pointer `front` dan `rear` sama-sama diarahkan ke awal array (`data`). Dengan kondisi ini, queue dianggap kosong karena belum ada elemen yang tersimpan. Saat `enqueue` pertama dilakukan, data akan dimasukkan pada posisi `rear`, kemudian `rear` akan maju satu langkah. Fungsi ini tidak memiliki kondisi tidak valid. Kompleksitas waktu O(1) dan kompleksitas ruang O(1).

Fungsi `isEmpty` digunakan untuk mengecek apakah queue kosong. Pengecekan dilakukan dengan membandingkan apakah `front == rear`. Jika kondisi ini terpenuhi, berarti tidak ada data dalam queue. Hal ini sesuai dengan kondisi awal setelah inisialisasi. Kompleksitas waktu O(1) dan kompleksitas ruang O(1).

Fungsi `isFull` digunakan untuk mengecek apakah queue sudah penuh. Hal ini dilakukan dengan memeriksa apakah jumlah elemen di dalam queue telah mencapai kapasitas maksimum, yaitu Ketika `(rear - data) == MAX`. Jika kondisi ini terpenuhi, maka tidak ada ruang lagi untuk menambahkan elemen baru. Fungsi ini hanya pengecekan sehingga tidak memiliki kondisi tidak valid. Kompleksitas waktu O(1) dan kompleksitas ruang O(1).

Fungsi `enqueue` digunakan untuk menambahkan data ke dalam queue. Langkah pertama adalah mengecek apakah queue penuh menggunakan `isFull`. Jika penuh, maka kondisi tersebut tidak valid dan program akan menghasilkan error (`throw`). Jika tidak penuh, maka nilai disimpan pada posisi yang ditunjuk oleh `rear`, kemudian dinaikkan satu posisi (`rear++`). Cara ini memastikan bahwa elemen baru selalu ditambahkan di bagian belakang queue. Dengan demikian, urutan FIFO tetap terjaga. Kompleksitas waktu O(1) dan kompleksitas ruang O(1).

Fungsi `dequeue` digunakan untuk menghapus elemen paling depan dari queue. Sebelum itu, dilakukan pengecekan apakah queue kosong menggunakan `isEmpty`. Jika kosong, maka kondisi tersebut tidak valid dan program akan menghasilkan error (`throw`). Jika tidak kosong, maka seluruh elemen dalam queue digeser satu posisi ke kiri menggunakan perulangan, sehingga elemen pertama terhapus. Setelah itu pointer `rear` dikurangi satu posisi (`rear--`). Cara ini sesuai dengan prinsip FIFO. Kompleksitas waktu O(n) karena melibatkan pergeseran elemen, dan kompleksitas ruang O(1).

Fungsi `front` digunakan untuk melihat nilai elemen paling depan tanpa menghapusnya. Sebelum mengambil nilai, dilakukan pengecekan apakah queue

kosong. Jika kosong, maka kondisi tersebut tidak valid dan akan menghasilkan error. Jika tidak kosong, nilai yang ditunjuk oleh pointer `front` dikembalikan. Fungsi ini tidak mengubah isi queue. Kompleksitas waktu O(1) dan kompleksitas ruang O(1).

Fungsi `back` digunakan untuk melihat nilai elemen paling belakang tanpa menghapusnya. Sebelum mengambil nilai, dilakukan pengecekan apakah queue kosong. Jika kosong, maka akan menghasilkan error. Jika tidak kosong, nilai yang dikembalikan adalah elemen terakhir dalam queue, yaitu berada pada posisi `rear - 1`. Fungsi ini juga tidak mengubah isi queue. Kompleksitas waktu O(1) dan kompleksitas ruang O(1).

SOAL 4: Si Palui dan Deretan Angka

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui memili sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n : jumlah elemen
- k : ukuran jendela
- A_i : elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Tabel 4 Source Code Prob2.cpp

```
#include <iostream>
#include "queue.h"
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;

    int arr[100];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    Queue q;
    init(&q);

    int jumlah = 0;

```

```
    if (k > n) k = n;

    for (int i = 0; i < k; i++) {
        enqueue(&q, arr[i]);
        jumlah += arr[i];
    }

    cout << jumlah;

    for (int i = k; i < n; i++) {
        int depan = front(&q);
        jumlah -= depan;
        dequeue(&q);

        enqueue(&q, arr[i]);
        jumlah += arr[i];

        cout << " " << jumlah;
    }

    cout << endl;

    return 0;
}
```

B. Output Program

```
D:\Algoritma & Struktur Data\Modul 2>g++ prob2.cpp queue.cpp -o prob2

D:\Algoritma & Struktur Data\Modul 2>prob2
5 3
1 2 3 4 5
6 9 12

D:\Algoritma & Struktur Data\Modul 2>prob2
5 5
1 2 3 4 5
15

D:\Algoritma & Struktur Data\Modul 2>prob2
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
```

Gambar 2. Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Program ini digunakan untuk menghitung jumlah elemen dalam suatu deretan angka dengan menggunakan konsep sliding window dan struktur data queue. Pada baris awal program terdapat `int n, k;` yang digunakan untuk menyimpan jumlah data dan ukuran window. Kemudian pada baris `cin >> n >> k;` program membaca banyaknya data serta jumlah elemen yang akan dijumlahkan dalam satu waktu. Nilai `k` ini menentukan ukuran window dalam proses perhitungan. Program jugaa melakukan pengecekan jika nilai `k` lebih besar dari `n`, maka `k` disamakan dengan `n`. Hal ini bertujuan untuk mencegah kesalahan saat mengakses array, karena ukuran window tidak boleh melebihi jumlah data yang tersedia.

Selanjutnya disediakan array `arr[100];` untuk menyimpan data input. Pada perulangan `for (int i = 0; i < n; i++)` dilakukan pembacaan data satu per satu menggunakan `cin >> arr[i];`, sehingga semua nilai tersimpan dalam array.

Setelah itu dibuat sebuah queue dengan `Queue q;`, lalu diinisialisasi menggunakan `init(&q);` agar berada dalam kondisi kosong dan siap digunakan.

Queue ini nantinya digunakan untuk menyimpan elemen-elemen yang berada dalam window.

Kemudian dibuat variabel `jumlah` untuk menyimpan total penjumlahan sementara. Pada perulangan pertama `for (int i = 0; i < k; i++)`, program memasukkan `k` elemen pertama ke dalam queue menggunakan `enqueue(&q, arr[i]);`, sekaligus menjumlahkannya ke dalam variabel `jumlah` dengan `jumlah += arr[i];`. Setelah itu, hasil penjumlahan pertama langsung ditampilkan menggunakan `cout << jumlah;`.

Selanjutnya pada perulangan `for (int i = k; i < n; i++)`, program mulai menggeser window. Nilai paling depan diambil menggunakan `front(&q);`, lalu dikurangkan dari total karena elemen tersebut akan dikeluarkan. Setelah itu dilakukan `dequeue(&q);` untuk menghapus elemen paling depan dari queue.

Kemudian elemen baru dimasukkan ke dalam queue menggunakan `enqueue(&q, arr[i]);`, dan nilainya ditambahkan ke dalam total dengan `jumlah += arr[i];`. Setelah proses ini, nilai `jumlah` merepresentasikan hasil penjumlahan window yang baru, lalu ditampilkan menggunakan `cout << " " << jumlah;`.

Sebagai contoh, jika diberikan input `n = 5, k = 3`, dan data `1 2 3 4 5`, maka pertama dijumlahkan `1 + 2 + 3 = 6`, lalu window digeser menjadi `2 + 3 + 4 = 9`, kemudian `3 + 4 + 5 = 12`, sehingga output yang dihasilkan adalah `6 9 12`.

Secara keseluruhan, program ini menggunakan queue untuk menyimpan elemen dalam window dan menghitung jumlah secara efisien tanpa harus menjumlahkan ulang dari awal setiap kali. Kompleksitas waktu dari program ini adalah O(n) karena setiap elemen hanya diproses satu kali, sedangkan kompleksitas ruangnya adalah O(k) karena queue hanya menyimpan maksimal k elemen.

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-snafisahh